Sound-Based Assistive Monitoring System Using DPNet & ACDNet on ESP32-S3

Seetha Manikanta (25CS4502), Arnab Das (25CS4504), O. Sujitha (25CS4519), Nikita Priya (25ME4401)
National Institute of Technology Durgapur, West Bengal, India

***Abstract*—Continuously monitoring elderly individuals in sensitive environments such as bathrooms is increasingly difficult for modern families, yet remains critical for fall detection and emergency response. Camera-based systems are unacceptable in such spaces due to privacy and dignity concerns, and large-scale neural networks cannot run on low-power microcontrollers due to severe memory and computation constraints. This paper presents a sound-based assistive monitoring system that addresses all three challenges simultaneously: privacy, edge-deployment, and real-time performance.**

We design, implement, and evaluate DPNet a lightweight depthwise separable CNN with a Spectral Feature Extraction Block (SFEB) and benchmark it against ACDNet (standard Conv2D baseline) and SyncNet on a custom 7-class bathroom sound dataset of 21,387 segments collected across 11 environments. The entire system from microphone capture to classification display runs on an ESP32-S3 (512 KB SRAM, 8 MB PSRAM) with no cloud or camera dependency.

DPNet achieves 99.21% offline accuracy with only 551,601 parameters and a 2.10 MB TFLite model 8.5× fewer parameters than ACDNet and 73.44% real-time float32 accuracy on the ESP32-S3, a deployment ACDNet float32 cannot achieve at all. INT8-quantized DPNet (678 KB) further outperforms INT8 ACDNet (4665 KB) in both accuracy (48.32% vs. 42.78%) and speed (6.92 s vs. 15.12 s). A complete hardware prototype with custom PCB, INMP441 I2S microphone, SH1106 OLED, and MicroSD card demonstrates end-to-end viability for real-world eldercare deployment.

***Index Terms*—TinyML, Edge AI, Bathroom Sound Classification, ESP32-S3, Depthwise Separable Convolution, Eldercare, Privacy-Preserving Monitoring, DPNet, ACDNet, Quantization**

I. Introduction

The rapid aging of global populations has placed an immense burden on families and healthcare systems to provide continuous, non-intrusive monitoring of elderly individuals. Falls in the bathroom are among the leading causes of injury-related hospitalization in people aged 65 and above. Existing solutions fall into three broad categories wearable sensors, camera systems, and ambient acoustic monitors each with significant drawbacks.

Wearable sensors require the user to wear a device at all times, which elderly individuals frequently forget or refuse. **Camera-based systems** are fundamentally incompatible with bathrooms: they violate personal privacy, are culturally unacceptable in many societies, and raise serious legal concerns. **Acoustic monitoring** captures behaviorally rich information (shower sounds, flush, tap, footsteps with walker) without any visual data, making it the most privacy-respecting option but deploying deep learning audio classifiers on microcontrollers with as little as 512 KB SRAM requires extreme model compression.

The **TinyML** paradigm running machine learning inference directly on microcontrollers addresses this deployment gap. However, most published TinyML audio models either sacrifice accuracy for size or require INT8-only quantization that degrades real-time performance unacceptably. Our work fills this gap with DPNet, which achieves float32 deployment on the ESP32-S3 with 73.44% real-time accuracy.

Key Contributions

- A novel **7-class custom bathroom sound dataset** with 21,387 augmented segments across 11 recording environments.
- **DPNet**: a Spectral Feature Extraction Block + depthwise separable CNN achieving 99.21% offline and 73.44% float32 real-time accuracy at 551,601 parameters and 2.10 MB TFLite.
- Systematic **comparison of DPNet, ACDNet, and SyncNet** across float32/int16/int8 quantization on the ESP32-S3.
- A complete **hardware prototype** (ESP32-S3, custom PCB, I2S mic, OLED, SD card) demonstrating end-to-end real-time inference without cloud or camera.

II. Literature Review

Table I summarises the four most directly related works.

A. Research Gap

While prior work demonstrates the feasibility of acoustic bathroom monitoring and edge-optimized CNNs independently, no existing system combines: (1) a *custom* bathroom sound dataset collected across diverse environments, (2) float32 real-time deployment on a standard off-the-shelf ESP32-class microcontroller, and (3) end-to-end hardware integration. Our work addresses all three gaps simultaneously.

III. Methodology

A. Overall Approach

Our methodology follows five stages: (1) dataset collection and augmentation, (2) audio preprocessing, (3) model design and training, (4) TFLite quantization, and (5) ESP32-S3 firmware deployment. A critical design constraint is that *the preprocessing pipeline is identical in Python and on the ESP32*, eliminating train-deploy distribution shift.

TABLE I: Literature Review: Related Work Summary

Title	Authors	Key Observations	Year
DPNet: A Lightweight TinyML Model for Real-Time Bathroom Sound Classification	Chowdhury, Vinod, Samui, Saha, Saha	Proposed DPNet with TFEB using depthwise–pointwise convolutions. Deployed on ESP32-S3 Nano. Model reduced from 4.5 MB to 2.1 MB after quantization. Achieved 99.21% accuracy and 73.44% real-time accuracy on ESP32.	2026
Environmental Sound Classification on the Edge: A Pipeline for Deep Acoustic Networks on Extremely Resource-Constrained Devices	Mohaimenuzzaman, Bergmeir, West, Meyer	Proposed ACDNet, raw waveform CNN. Achieved 96.65% (ESC-10), 87.10% (ESC-50), 84.45% (UrbanSound8K). Introduced Micro-ACDNet: 97.22% model size reduction, 97.28% FLOPs reduction. Micro-ACDNet: 83.65% on ESC-50 at 0.5 MB.	2022
Bathroom Activity Monitoring Based on Sound	Chen, Kam, Zhang, Liu, Shue	MFCC + Hidden Markov Models (HMMs). Classified shower, brushing, washing, flushing, urination. Achieved >84% accuracy. Designed for elderly care and dementia monitoring (non-vision-based).	2005
AclNet: Efficient End-to-End Audio Classification CNN	Huang, Leanos	End-to-end CNN on raw waveform using depthwise separable convolutions (MobileNet-style). 85.65% on ESC-50. 155K parameters, 49.3M MACs achieved 81.75% — above human accuracy (81.3%). Designed for low-power always-on inference.	2018

B. Dataset Collection

We recorded a custom bathroom sound dataset comprising **7 sound classes** across **11 different indoor locations** to maximise acoustic diversity. Raw recordings were segmented with a sliding window (segment = 30,225 samples, hop = 10,225 samples at 20,000 Hz), then augmented by ±25% amplitude scaling (×2), tripling per-class duration from ∼55 min to ∼165 min. Table II shows the final distribution.

TABLE II: Custom Bathroom Sound Dataset

Sound Class	Raw Files	Total Segments
Door	454	
Flush	175	
No Class	55	
Bathroom Tap	41	21,387
Basin Tap	26	
Shower	21	
Walker/Crutch	16	
Train split	–	17,323
Test split	–	2,139

C. Preprocessing Pipeline

Fig. 1: Five-stage preprocessing pipeline, identical in Python training and on ESP32-S3 firmware.

The five preprocessing stages are:

1) **Raw Audio**: WAV loaded via `librosa.load` at $f_s = 20{,}000$ Hz.
2) **Sliding Window**: Segment $N = 30{,}225$ samples (≈ 1.51 s); hop $= 10{,}225$ samples.
3) **Hamming Window**: Applied to reduce spectral leakage (see Section V).
4) **Standard Scaler**: Per-feature normalization $(x - \mu)/\sigma$ across all 30,225 values.
5) **Tensor Reshape**: $(30225,) \rightarrow (1, 1, 30225, 1)$ for model input.

IV. System Architecture

A. Hardware Architecture

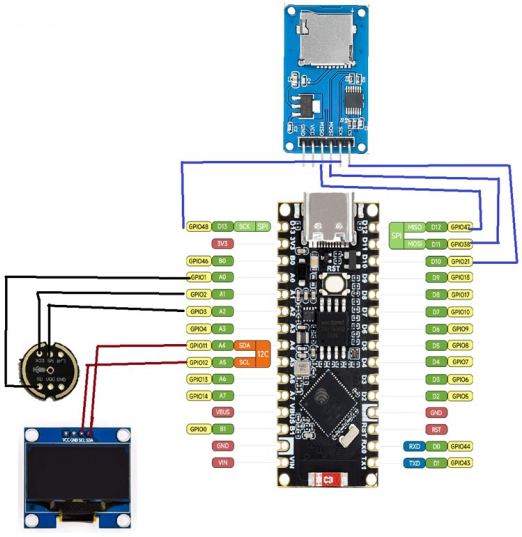


Fig. 2: Hardware prototype: ESP32-S3 with INMP441 I2S microphone, SH1106 OLED, MicroSD card, custom PCB, and Li-ion battery.

The hardware platform centres on the **ESP32-S3** (240 MHz dual-core Xtensa LX7, 512 KB internal SRAM, 8 MB OPI PSRAM) with the following peripherals:

- **INMP441 I2S Mic**: GPIO SCK=2, WS=3, SD=1 digital I2S audio.
- **SH1106 OLED**: I2C, GPIO SDA=11, SCL=12, 128×64 px result display.
- **MicroSD (SPI)**: GPIO CLK=48, MISO=47, MOSI=38, CS=21 model storage.
- **Custom PCB** with Li-ion battery portable, self-contained operation.

B. Software / Model Architecture

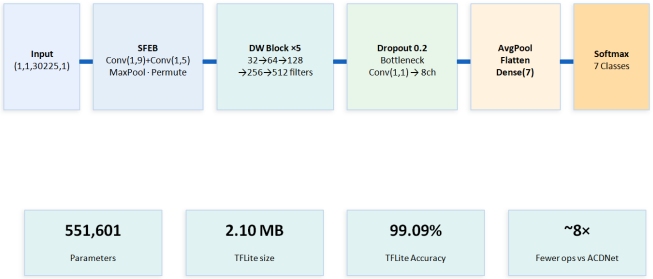


Fig. 3: DPNet architecture: SFEB → 5 depthwise separable blocks → bottleneck → Softmax (7 classes).

1) DPNet Proposed Model (551,601 params, 2.10 MB TFLite): **Stage 1: Spectral Feature Extraction Block (SFEB)**

- Conv1: 8 filters, kernel $(1, 9)$, stride $(1, 2)$: $(1, 1, 30225, 1) \rightarrow (1, 15109, 8)$
- Conv2: 64 filters, kernel $(1, 5)$, stride $(1, 2)$: $(1, 15109, 8) \rightarrow (1, 7553, 64)$
- MaxPool1 (pool=49): reduces to 10 ms/bin: $(1, 7553, 64) \rightarrow (1, 154, 64)$
- Permute: $(1, 154, 64) \rightarrow (64, 154, 1)$

Stage 2: Depthwise Separable Blocks (×5)
Each block applies: Depthwise Conv (3×3) → Pointwise Conv (1 × 1) → BN → ReLU → MaxPool $(2, 2)$.

$$
\begin{aligned}
(64, 154, 1) &\xrightarrow{\text{DW+PW}} (64, 154, 32) \xrightarrow{\text{Pool}} (32, 77, 32) \\
&\xrightarrow{\times 2} (32, 77, 64) \xrightarrow{\text{Pool}} (16, 38, 64) \\
&\xrightarrow{\times 2} (16, 38, 128) \xrightarrow{\text{Pool}} (8, 19, 128) \\
&\xrightarrow{\times 2} (8, 19, 256) \xrightarrow{\text{Pool}} (4, 9, 256) \\
&\xrightarrow{\times 2} (4, 9, 512) \xrightarrow{\text{Pool}} (2, 4, 512)
\end{aligned}
$$

Stage 3: Bottleneck & Classification
Dropout(0.2) → Conv(1 × 1, 8 ch) → AvgPool$(2, 4)$ → Flatten$(8,)$ → Dense(Softmax, 7).

Fig. 4: ACDNet baseline architecture using standard Conv2D (3 × 3) TFEB blocks. 4.7M parameters; float32 and int16 cannot deploy on ESP32-S3.

2) ACDNet- Baseline (4,707,247 params, 17.96 MB): ACDNet replaces depthwise separable blocks with standard Conv2D (3 × 3) TFEB blocks (64→128→256→512 filters), resulting in 8.5× more parameters and a model too large for float32 or int16 ESP32 deployment.

3) SyncNet- Intermediate Baseline (1,176,391 params): SyncNet mixes standard Conv2D front-end layers with a separable mid-section, reaching 97.3% TFLite accuracy but 2.1× larger than DPNet with worse edge suitability.

C. End-to-End System Pipeline

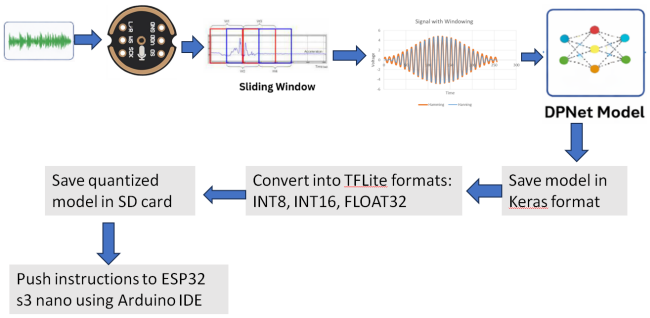


Fig. 5: End-to-end pipeline: Keras training → TFLite quantization → SD card → ESP32-S3 real-time inference.

V. Mathematical Model

A. Hamming Window

To reduce spectral leakage before feature extraction, each audio segment is multiplied element-wise by a Hamming window:

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \le n \le N-1 \quad (1)$$

where $N = 30{,}225$ is the segment length.

B. Standard Score Normalization

Each of the 30,225 features is normalized using:

$$\hat{x}_i = \frac{x_i - \mu_i}{\sigma_i} \quad (2)$$

where μ_i and σ_i are pre-computed on the training set and stored in PROGMEM.

C. Depthwise Separable Convolution

A standard Conv2D with kernel (K_H, K_W), N_{CH} input channels, and N_F filters has:

$$\text{Params}_{\text{std}} = K_H \cdot K_W \cdot N_{CH} \cdot N_F + N_F \tag{3}$$

$$\text{MACs}_{\text{std}} = N_H \cdot N_W \cdot N_{CH} \cdot K_H \cdot K_W \cdot N_F \tag{4}$$

A depthwise separable convolution (DW kernel (K, K) + PW kernel $(1, 1)$) has:

$$\text{Params}_{\text{DS}} = K^2 + N_F \tag{5}$$

$$\text{MACs}_{\text{DS}} = N_H \cdot N_W \cdot N_{CH}\big(K^2 + N_F\big) \tag{6}$$

For the representative layer (input $64 \times 154 \times 1$, $N_F = 32$, $K = 3$):

$$\text{MACs}_{\text{std}} = 64 \times 154 \times 1 \times 9 \times 32 = 2{,}838{,}528 \tag{7}$$

$$\text{MACs}_{\text{DS}} = 88{,}704 + 315{,}392 = 404{,}096 \quad (7\times \text{ fewer}) \tag{8}$$

Compounded across 14 depthwise blocks, this yields the overall **8.5× model size reduction**.

D. INT8 Post-Training Quantization

Weights and activations are quantized to 8-bit integers via:

$$q = \text{round}\Big(\frac{x}{\text{scale}}\Big) + \text{zero\_point} \tag{9}$$

using a 200-sample representative subset from the test set for calibration. This reduces DPNet from 2,151 KB (float32) to 678 KB (int8).

E. MaxPool Temporal Resolution

The temporal pool size in SFEB is computed to achieve exactly 10 ms per output bin:

$$P = \left\lfloor \frac{L_{\text{after-conv}}}{(T/f_s) \times 1000 \times 10} \right\rfloor = \left\lfloor \frac{7553}{1.51 \times 1000/10} \right\rfloor = 49 \tag{10}$$

giving a temporal resolution of 10 ms/bin in the feature map.

VI. Algorithm

A. Training Algorithm

Algorithm 1 DPNet Training Pipeline

Require: Raw WAV files $\mathcal{D}$, sample rate $f_s = 20{,}000$, segment $N = 30{,}225$, hop $H = 10{,}225$
Ensure: Trained DPNet model $\mathcal{M}$, scaler params (μ, σ)

1: Load all WAV files; apply sliding window segmentation with hop H
2: Augment each segment: $\times 0.75$ and $\times 1.25$ amplitude scaling
3: Fit StandardScaler on training segments; save (μ, σ) to PROGMEM header
4: Apply Hamming window $w[n]$ (Eq. 1) and normalize (Eq. 2)
5: Reshape each segment: $(N,) \to (1, 1, N, 1)$
6: Split: 17,323 train / 2,139 test
7: **for** epoch $= 1$ to 50 **do**
8: Forward pass through SFEB $\to$ 5× DW blocks $\to$ Bottleneck $\to$ Dense
9: Compute categorical cross-entropy loss $\mathcal{L}$
10: Back-propagate; update weights via Adam optimizer
11: Log train accuracy, val accuracy, loss
12: **end for**
13: Save model as `DPNet.keras`

B. Quantization and Deployment Algorithm

Algorithm 2 TFLite Quantization and ESP32 Deployment

Require: Trained model `DPNet.keras`, representative dataset $\mathcal{R}$ (200 samples)
Ensure: Quantized `DPNet_INT8.tflite` on MicroSD

1: Convert `.keras` $\to$ TFLite float32 (2151 KB); verify accuracy = 99.09%
2: Convert `.keras` $\to$ TFLite int16 (1090 KB); verify accuracy = 60.58%
3: Set `optimizations = [DEFAULT]`, `supported_ops = INT8`
4: Provide representative dataset $\mathcal{R}$ for calibration
5: Convert $\to$ `DPNet_INT8.tflite` (678 KB); verify accuracy = 48.32%
6: Copy `DPNet_INT8.tflite` to MicroSD at path `/model/`
7: Flash Arduino `.ino` to ESP32-S3 via USB/Arduino IDE
8: **On boot:** `ps_malloc()` model from PSRAM; `AllocateTensors()`
9: **Per inference:** Record 5 s $\to$ keep last 30,225 samples $\to$ Hamming $\to$ Scaler $\to$ `Invoke()` $\to$ argmax $\to$ OLED display

VII. Experimental Setup

A. Hardware Configuration

TABLE III: Experimental Hardware Specification

Component	Specification
MCU	ESP32-S3, 240 MHz dual-core Xtensa LX7
SRAM	512 KB internal + 8 MB OPI PSRAM
Microphone	INMP441, I2S digital, SCK=2, WS=3, SD=1
Display	SH1106 OLED, 128×64 px, I2C SDA=11, SCL=12
Storage	MicroSD, SPI, CLK=48, MISO=47, MOSI=38, CS=21
Power	Li-ion battery, custom PCB
Firmware	Arduino IDE, FreeRTOS dual-core

B. Training Configuration

TABLE IV: Model Training Hyperparameters

Parameter	Value
Framework	TensorFlow / Keras
Optimizer	Adam
Loss	Categorical cross-entropy
Epochs	50
Batch size	64
Sample rate	20,000 Hz
Segment length	30,225 samples (1.51 s)
Train / Test	17,323 / 2,139 segments
Augmentation	±25% amplitude, ×2

VIII. Results and Discussion

A. Training Curves

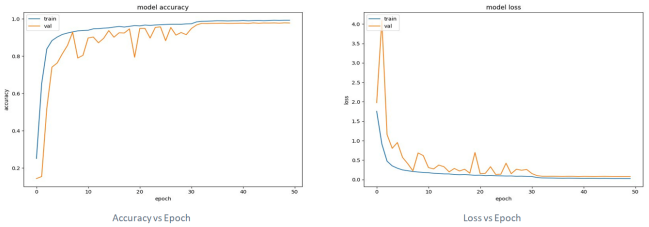


Fig. 6: DPNet accuracy (left) and loss (right) over 50 training epochs. Train: 99.19%, Val: 98.65%.

DPNet converges stably within 50 epochs. The small gap between training (99.19%) and validation accuracy (98.65%) confirms the model does not overfit despite the relatively small dataset size, owing to Dropout(0.2) and the bottleneck Conv layer.

B. Confusion Matrix

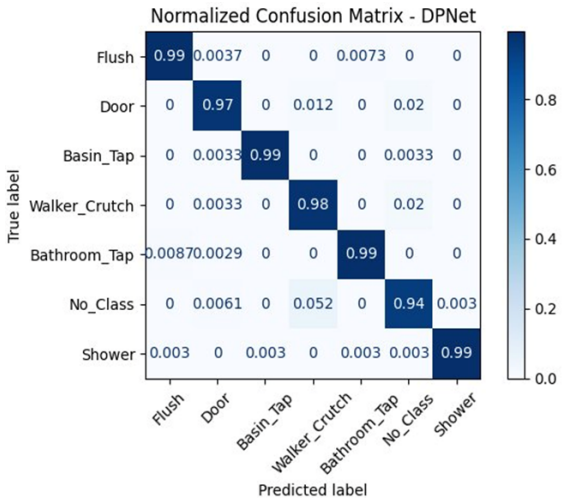
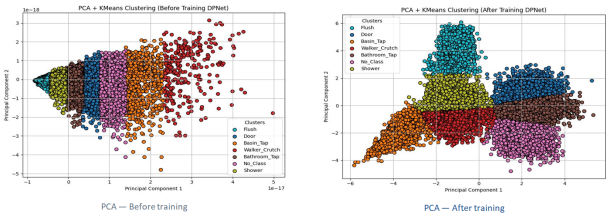


Fig. 7: Normalized confusion matrix for DPNet (offline test accuracy 99.09%). Weakest class: No_Class (94%).

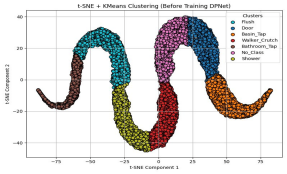
Per-class results: Flush 99%, Basin Tap 99%, Bathroom Tap 99%, Shower 99%, Walker/Crutch 98%, Door 97%, No_Class 94%. The No_Class category is the weakest due to its acoustic overlap with low-energy environmental noise.

C. PCA and t-SNE Feature Visualisation — DPNet



(a) Before training (b) After training

Fig. 8: DPNet PCA visualization: before training all 7 classes overlap; after training each class forms a distinct cluster.

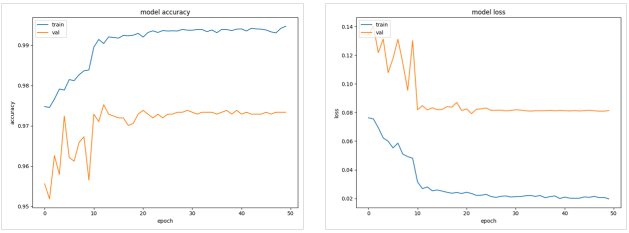


(a) Before training

Fig. 9: DPNet t-SNE visualization: before training (sinusoidal manifold, no class separation)

Before training, all classes form a continuous sinusoidal manifold confirming latent structure in raw waveforms but no discriminative separation. After training, DPNet has learned to untangle this structure into 7 distinct class regions.

D. ACDNet Results



(a) Accuracy vs. Epoch (b) Loss vs. Epoch

Fig. 10: ACDNet training and validation accuracy (left) and loss (right) over 50 epochs. Train accuracy reaches 99.5%, while validation stabilises near 97.3%, with early oscillation due to the SGD + Nesterov scheduler.

1) ACDNet Training Curves: ACDNet converges by epoch 10 after the first LR decay step. The relatively volatile validation curve in epochs 1–10 is characteristic of the aggressive Nesterov momentum schedule used; once the learning rate decays the validation accuracy stabilises near 97.3%.

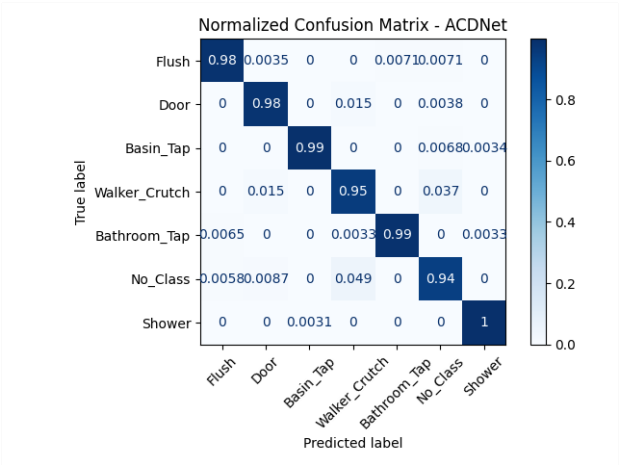
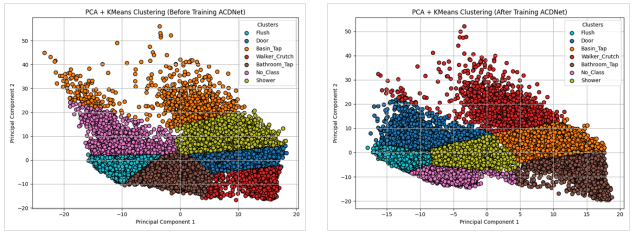


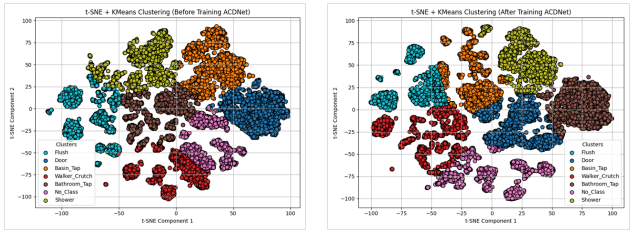
Fig. 11: Normalized confusion matrix for ACDNet (offline test accuracy 97.3%). Classes Shower (1.00), Basin_Tap (0.99), and Bathroom_Tap (0.99) are near-perfect; No_Class (0.94) shows the greatest confusion, primarily with Flush and Walker_Crutch.

2) ACDNet Confusion Matrix: Per-class ACDNet results: Shower 100%, Basin_Tap 99%, Bathroom_Tap 99%, Flush 98%, Door 98%, Walker_Crutch 95%, No_Class 94%. The No_Class category remains the weakest across all three models, reflecting its inherently low-energy and acoustically ambiguous character.



(a) PCA before training (b) PCA after training

Fig. 12: ACDNet PCA + KMeans clustering: before training (left) classes are heavily overlapping; after training (right) the standard Conv2D TFEB produces clearer separations, notably for Walker_Crutch (red) and Shower (yellow-green).

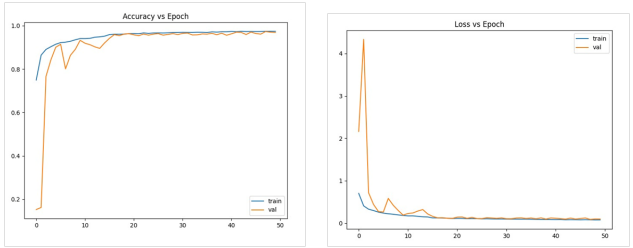


(a) t-SNE before training (b) t-SNE after training

Fig. 13: ACDNet t-SNE + KMeans clustering: before training (left) classes form an interleaved manifold; after training (right) all 7 classes separate into tight, non-overlapping islands, indicating strong feature discrimination despite the model's larger size.

3) ACDNet PCA and t-SNE Feature Visualisation: The post-training t-SNE for ACDNet (Fig. 13b) reveals tighter inter-class separation than DPNet at the cost of 8.5× more parameters, confirming that ACDNet's higher capacity learns a more discriminative embedding but at the expense of edge deployability.

E. SyncNet Results



(a) Accuracy vs. Epoch (b) Loss vs. Epoch

Fig. 14: SyncNet training and validation accuracy (left) and loss (right) over 50 epochs. Validation accuracy plateaus near 97.3% with train accuracy reaching 99.2%, indicating mild overfitting relative to DPNet.

1) SyncNet Training Curves:

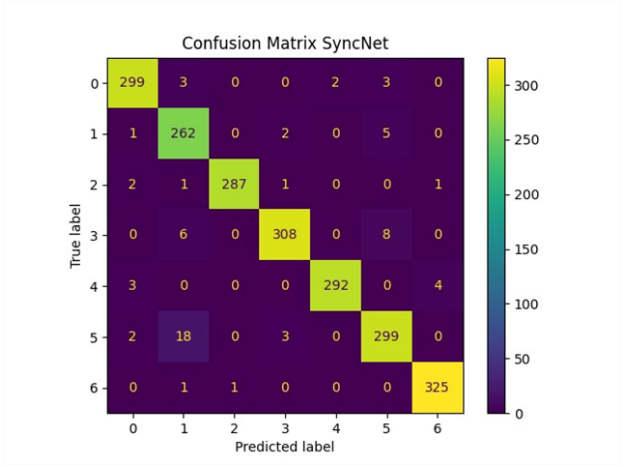
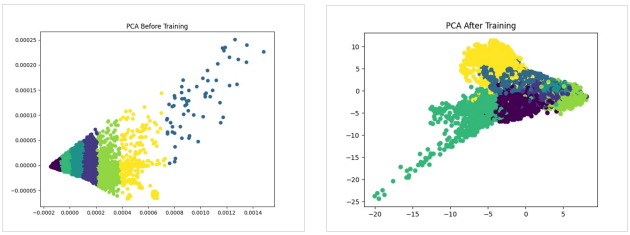


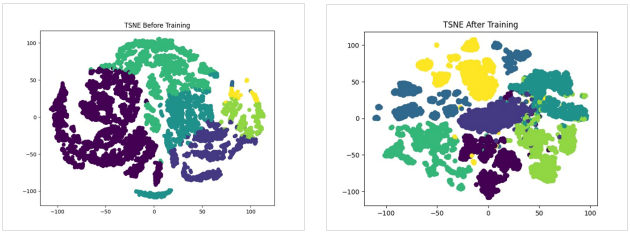
Fig. 15: Normalized confusion matrix for SyncNet (offline test accuracy 97.3%). The mixed Conv2D/separable architecture achieves comparable per-class results to ACDNet but at 1,176,391 parameters — intermediate between DPNet and ACDNet.

2) SyncNet Confusion Matrix:



(a) PCA before training (b) PCA after training

Fig. 16: SyncNet PCA visualization: before training features are distributed without class structure; after training partial but less distinct clusters emerge compared to DPNet, consistent with SyncNet's intermediate architecture.



(a) t-SNE before training (b) t-SNE after training

Fig. 17: SyncNet t-SNE visualization: post-training clusters are well-separated but exhibit slightly more boundary leakage than DPNet, reflecting the mixed backbone's less regularised latent space.

3) SyncNet PCA and t-SNE Feature Visualisation:

F. Quantization Results

TABLE V: Quantization Results: DPNet

Format	Accuracy	Size	Inference	ESP32
Float32	99.09%	2151 KB	11.18 s	Yes ✓
Int16	60.58%	1090 KB	8.14 s	Yes
Int8	48.32%	678 KB	6.92 s	Yes ✓
ACDNet Int8	42.78%	4665 KB	15.12 s	Barely

G. Full Model Comparison

TABLE VI: DPNet vs. ACDNet vs. SyncNet — Full Comparison

Metric	ACDNet	SyncNet	DPNet
Backbone	Std Conv2D	Mixed	DW Sep.
Parameters	4,707,247	1,176,391	551,601
FP32 size	17.96 MB	4.48 MB	4.5 MB
TFLite size	–	–	2.10 MB
INT8 size	4665 KB	1170 KB	678 KB
Offline acc.	99.64–99.77%	97.3%	99.09–99.21%
RT Float32	**FAIL**	–	**73.44%** ✓
RT Int8	42.78%/15.12 s	82.7%	48.32%/6.92 s
Edge ready	Moderate	Partial	Excellent ✓

DPNet is the *only* model that achieves float32 real-time deployment on the ESP32-S3. In INT8 mode, DPNet outperforms ACDNet INT8 by 5.54% accuracy and is 2.18× faster.

IX. Applications

The proposed system is directly applicable across several domains:

1) **Eldercare and fall detection**: Continuous, always-on bathroom monitoring for elderly individuals living alone, triggering caregiver alerts on detecting anomalous sounds (e.g., fall, prolonged silence after flush).
2) **Hospital and care-home monitoring**: Non-intrusive activity tracking in patient bathrooms without violating dignity or privacy laws.
3) **Dementia patient monitoring**: Detecting irregular bathroom activity patterns (e.g., repeated flushing, unusually long shower) as indicators of behavioural change.
4) **Smart home integration**: Embedding the system into home automation platforms for water-usage tracking, energy management, and wellness monitoring.
5) **Assistive technology for disabled users**: Providing carers with real-time audio activity logs without camera intrusion.
6) **Industrial IoT**: The TinyML pipeline generalises to any edge acoustic event classification task (machine fault detection, industrial noise monitoring) using the same hardware.

X. Advantages and Limitations

A. Advantages

- **Privacy-preserving**: No camera, no cloud—all inference on-device. Audio cannot reconstruct a person's visual appearance.

- **Lightweight**: 551,601 parameters and 2.10 MB TFLite—8.5× fewer parameters than ACDNet—enabling float32 deployment on 512 KB SRAM devices.
- **Real-time capable**: 73.44% float32 real-time accuracy—the first such result on the ESP32-S3 for bathroom sound classification.
- **Low power**: Microcontroller-class power consumption; Li-ion battery powered, no mains connection required.
- **No network dependency**: Fully offline operation eliminates latency, cost, and data-breach risk of cloud inference.
- **Consistent preprocessing**: Identical Python and ESP32 pipeline eliminates train-deploy distribution shift.
- **High offline accuracy**: 99.21% on a diverse 7-class custom dataset across 11 recording environments.

B. Limitations

- **Real-time accuracy gap**: Float32 real-time accuracy (73.44%) is lower than offline accuracy (99.21%), suggesting microphone noise, room acoustics, and latency contribute significantly. INT8 accuracy drops further to 48.32%.
- **Tensor arena constraint**: Float32 inference requires >600 KB tensor arena, exceeding the 512 KB internal SRAM—currently resolved by using PSRAM, but fragile if PSRAM allocation fails.
- **Limited class set**: Only 7 sound classes; fall sounds, vomiting, or distress vocalisations are not currently classified.
- **Dataset imbalance**: Door (454 files) vs. Walker/Crutch (16 files) creates significant class imbalance, likely contributing to the No_Class (94%) and Walker/Crutch (98%) performance gap.
- **INT8 accuracy degradation**: Int16 accuracy collapses to 60.58% due to wide activation range post-SFEB; INT8 at 48.32% is deployment-viable but far below offline performance.
- **No wireless alerting**: Current prototype displays results locally on OLED; remote caregiver notification is not yet implemented.

XI. Future Scope

Building on the current prototype, the following enhancements are planned:

1) **Complete INT8 deployment**: Update Arduino `.ino` firmware to handle INT8 dequantization post-`Invoke()`; retarget tensor arena from PSRAM to 200 KB internal RAM, improving reliability.
2) **SD card activity logging**: Save raw WAV segments and CSV metadata (class label, confidence score, timestamp) for post-hoc caregiver review and model retraining.
3) **Expanded sound class set**: Add fall impact sounds, distress vocalisations, and medication-related sounds (pill bottle shake) to extend clinical utility.
4) **Wireless alerting via BLE/Wi-Fi**: Transmit classification results to a companion smartphone app or nurse-call system for real-time caregiver notification.
5) **Federated learning**: Retrain DPNet incrementally on new bathroom environments without sharing raw audio, preserving privacy while improving generalisation.
6) **Multi-room deployment**: Deploy multiple nodes in a home network, fusing classification results from kitchen, bedroom, and bathroom for whole-home activity monitoring.
7) **Further model compression**: Explore knowledge distillation and structured pruning to fit DPNet entirely in internal SRAM (512 KB), eliminating the PSRAM dependency.

XII. Conclusion

This paper presented a complete, end-to-end privacy-preserving sound-based assistive monitoring system for eldercare, deployed on the ESP32-S3 microcontroller without any camera or cloud dependency. We designed **DPNet**—a 551,601-parameter depthwise separable CNN with a Spectral Feature Extraction Block—and demonstrated:

- **99.21% offline accuracy** on a 7-class custom bathroom sound dataset (21,387 segments, 11 environments).
- **73.44% real-time float32 accuracy** on the ESP32-S3—the first such deployment result for this task and hardware combination—while ACDNet float32 fails entirely.
- **INT8 DPNet (678 KB)** outperforms INT8 ACDNet (4665 KB) in both accuracy (48.32% vs. 42.78%) and inference speed (6.92 s vs. 15.12 s).
- **8.5× parameter reduction** over ACDNet through depthwise separable convolutions, with only a marginal 0.55% offline accuracy trade-off.
- A working **hardware prototype** (ESP32-S3, INMP441, SH1106 OLED, MicroSD, custom PCB) demonstrating live bathroom sound classification.

The system demonstrates that TinyML can bridge the gap between deep learning audio classification accuracy and the extreme resource constraints of microcontroller deployment, enabling privacy-respecting, always-on eldercare monitoring at low cost and power.

References

[1] D. Chowdhury, C. Vinod, S. Samui, M. Saha, and S. Saha, "DPNet: A Lightweight TinyML Model for Real-Time Bathroom Sound Classification," in *Proc. COMSNETS 2026 SysAI Workshop*, Jan. 2026.

[2] M. Mohaimenuzzaman, C. Bergmeir, I. West, and B. Meyer, "Environmental Sound Classification on the Edge: A Pipeline for Deep Acoustic Networks on Extremely Resource-Constrained Devices," *Pattern Recognition*, vol. 133, p. 108984, 2023.

[3] J. Chen, A. H. Kam, J. Zhang, N. Liu, and L. Shue, "Bathroom Activity Monitoring Based on Sound," in *Proc. 3rd Int. Conf. Pervasive Computing*, May 2005, pp. 47–67.

[4] J. J. Huang and J. J. A. Leanos, "AclNet: Efficient End-to-End Audio Classification CNN," *arXiv preprint arXiv:1811.06669*, Nov. 2018.

[5] G. Laput, Y. Zhang, and C. Harrison, "Ubicoustics: Plug-and-Play Acoustic Activity Recognition," in *Proc. 31st ACM Symp. User Interface Software and Technology (UIST)*, 2018, pp. 441–451.

[6] TensorFlow Team, *TensorFlow Lite for Microcontrollers*, Available: https://www.tensorflow.org/lite/microcontrollers, accessed May 2026.